

COMILLAS
UNIVERSIDAD PONTIFICIA
ICAI ICADE CIHS

Sprint H4 y H5

Grupo: Alberto Rey, Adrián Calderón y Jorge Montoyo

Ingeniería del Dato
Máster Universitario en Inteligencia Artificial Aplicada (MUIAAP)

Índice

INTRODUCCIÓN.....**3**

METODOLOGÍA.....**4**

RESULTADOS.....**6**

CONCLUSIÓN.....**13**

Introducción

Contexto. Este sprint forma parte de BIODATAPHARMA, el sistema que automatiza la recolección, ordenación y análisis de datos de medicamentos a partir de la API de CIMA, las fichas técnicas y el Nomenclátor del Ministerio de Sanidad. A nuestro grupo le corresponde la diabetes. En las historias anteriores consolidamos un dataset de 1569 medicamentos y 32 columnas.

Descripción del problema y objetivos. Ese dataset todavía no se podía usar en un modelo: tenía nulos, tipos de datos incorrectos y una columna de dosis con 167 formatos distintos. Además, no sabíamos qué tipos de medicamentos contenía. Los objetivos del sprint son dos:

- **HU-4:** analizar los datos con gráficas, limpiarlos, escribir un informe con las conclusiones y obtener un CSV limpio.
- **HU-5:** entrenar un modelo K-Means con scikit-learn, analizar y poner nombre a los grupos, y obtener un CSV con la clasificación de cada medicamento.

Justificación. La calidad de un modelo depende de la calidad de sus datos, por lo que la limpieza es un paso necesario. Agrupar los medicamentos permite entender un catálogo de cientos de referencias sin revisarlas una a una, y da unos perfiles que se pueden explicar a personas que no conocen el modelo.

Metodología

Entorno de desarrollo

Hemos trabajado en cuadernos de Jupyter, con un entorno virtual creado con uv y estas versiones:

Herramienta	Versión	Para qué la usamos
Python	3.10.21	Lenguaje del proyecto
pandas	2.3.3	Cargar, limpiar y guardar los datos
matplotlib	3.10.9	Histogramas, gráficas de líneas, barras y dispersión
seaborn	0.13.2	Boxplots
scikit-learn	1.7.2	Escalado, K-Means, silhouette y PCA

Nos apoyamos en IA generativa para estudiar los casos de la columna de dosis y para interpretar los tipos de medicamentos de cada grupo, como se indica en los cuadernos `h4.ipynb` y `h5.ipynb`.

Diseño de la solución

La solución es un flujo de dos pasos, en el que la salida de la HU-4 es la entrada de la HU-5:

Cuaderno	Entrada	Salida
HU-4: `h4.ipynb`	`HU3_medicamentos_diabetes.xlsx` (1569 filas, 32 columnas)	`HU4_medicamentos_diabetes_limpio.csv` (827 filas, 28 columnas)
HU-5: `h5.ipynb`	`HU4_medicamentos_diabetes_limpio.csv`	`HU5_medicamentos_diabetes_clusters.csv` (827 filas, 30 columnas)

HU-4: limpieza y análisis exploratorio

- **Filas:** quitamos las 742 que no cruzan con el Nomenclátor, porque no tienen precio.
- **Columnas eliminadas:** el precio de referencia (cuando existe es igual al de venta), el código nacional repetido, las columnas con un solo valor y la URL de la ficha técnica.
- **Nulos:** `estado_rev` (88 % de nulos) y `url_foto_materiales` pasan a columnas 0/1. Las 3 filas sin sección 4.4 quedan con 0 palabras.
- **Tipos:** `nregistro` y `cn` pasan a texto, `estado_aut` a fecha y `tratamiento_larga_duracion` a 1/0.
- **Categorías:** agrupamos la forma farmacéutica en 4 categorías y la vía de administración en 3.
- **Dosis:** dejamos todas con el formato "número mg" y creamos `dosis_numero`, `dosis_numero_2` y `dosis_por_ml`.
- **Outliers:** los detectamos con el método IQR y boxplots. No eliminamos ninguno, porque los valores altos revisados son reales.
- **EDA:** analizamos las variables numéricas con boxplots e histogramas.

HU-5: modelo K-Means

- **Columnas:** usamos 13 columnas numéricas. Creamos `antiguedad_anios` y `es_inyectable`, y dejamos fuera los códigos, los textos y las columnas repetidas o casi constantes.

- **Escalado:** aplicamos `StandardScaler` para que todas las columnas tengan media 0 y desviación típica 1. K-Means usa distancias y, sin escalar, mandarían las columnas con números grandes.
- **Número de grupos:** probamos K de 2 a 10 con el método del codo y el coeficiente silhouette, y elegimos K = 6.
- **Modelo final:** `KMeans(n_clusters=6, random_state=42, n_init=50)`.
- **Interpretación:** analizamos las medias y los principios activos de cada grupo para ponerle nombre. Usamos PCA solo para dibujar los grupos en dos dimensiones.

Pruebas realizadas

Los cuadernos incluyen comprobaciones que se pueden repetir al ejecutarlos:

Comprobación	Resultado
`cn` y el código del Nomenclátor son iguales antes de eliminar uno	True
Insulinas sin equivalencia de UI a mg	0
Dosis que no acaban en "mg" o que aún tienen una barra	0 y 0
Nulos en el CSV de la HU-4 y en las columnas del modelo	0 y 0

Además, hicimos tres revisiones aparte, que no están en los cuadernos:

- **Dosis:** comparamos cada dosis final con la que aparece en el nombre del medicamento, lo que sirvió para corregir a mano 5 filas (Byetta, Ristaben y Glyxambi).
- **Estabilidad:** repetimos el modelo con otras 7 semillas y salieron los mismos grupos (índice de Rand ajustado de 1 en todas).
- **CSV de la HU-5:** lo volvimos a leer y tiene 827 filas, 30 columnas y 0 nulos.

Resultados

Descripción de los resultados

HU-4: dataset limpio

Medida	Antes	Después
Filas	1569	827
Columnas	32	28
Formatos de dosis	167	1 (47 valores distintos)
Nulos	Varias columnas con nulos	0

La mayoría de los precios son bajos, pero hay unos pocos muy altos: la mediana es de 11,18 €, la media de 20,30 € y el máximo de 133,47 €. Las fichas técnicas también cambian mucho: la mediana es de 2 tablas, pero algunas llegan a 32. La sección 4.4 tiene una mediana de 1068 palabras y un máximo de 3692.

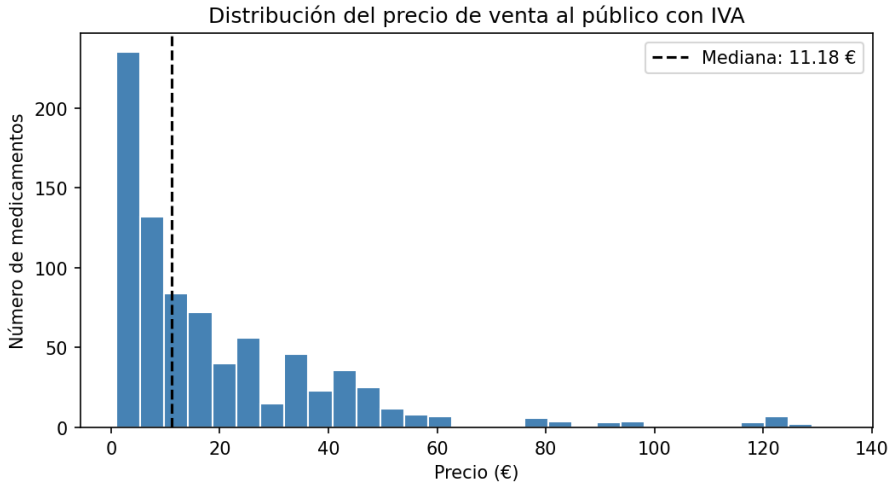


Figura 1. Distribución del precio de venta al público con IVA.

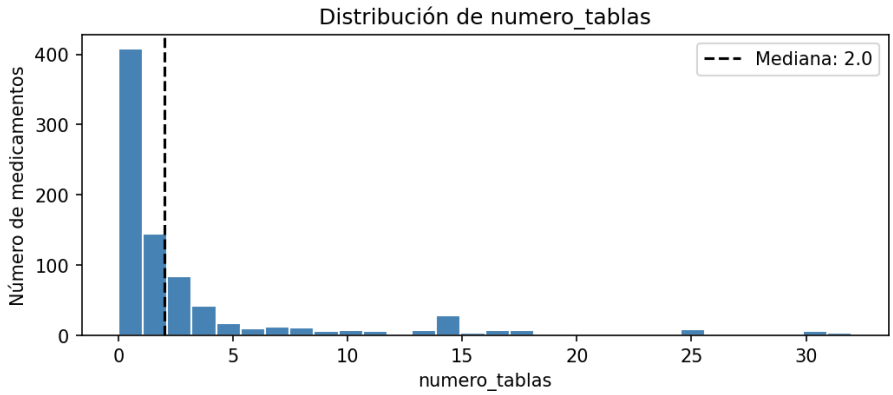


Figura 2. Distribución del número de tablas de las fichas técnicas.

Con el método IQR encontramos estos outliers, que hemos mantenido:

Columna	Outliers	Qué son
`numero_tablas`	121	Fichas técnicas con muchas tablas, como las de dabigatrán
`palabras_seccion_4_4`	62	Fichas técnicas largas, como las de simvastatina
`dosis_numero`	52	Sobre todo metformina (850 y 1000 mg) e irbesartán (300 mg)
`precio_pvp_iva`	36	Todos son medicamentos no genéricos
`indicador_riesgo_severo`	4	Fichas con muchas menciones de "grave"
`num_excipientes`	1	MAPOLI, con 9 excipientes

HU-5: perfiles de medicamentos

Elegimos K = 6 porque su silhouette (0,292) es mejor que el de K = 5 (0,286) y K = 7 (0,276). Con K = 8 solo se iguala y con 9 y 10 sube muy poco (hasta 0,306), a cambio de tener más grupos y más difíciles de explicar. La inercia baja de forma bastante suave, así que el método del codo no da un punto claro.

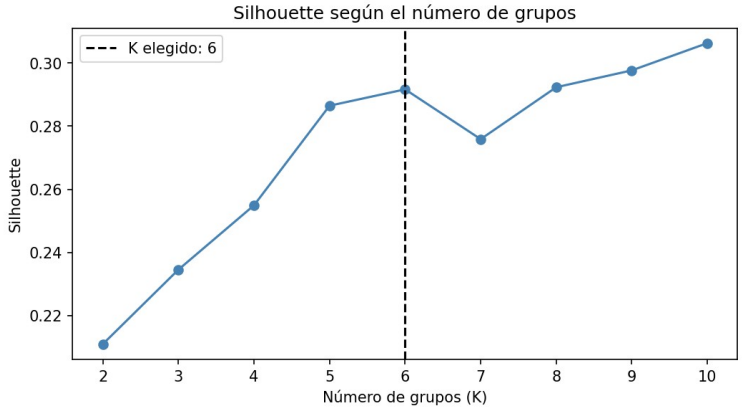


Figura 3. Silhouette para cada valor de K.

El modelo ha encontrado estos 6 perfiles. Los nombres son nuestra interpretación de cada grupo:

Perfil	Medicamentos	Mediana del precio	Lo que lo define
Comprimidos genéricos de uso general	335	13,27 €	Valores cerca de la media y 84 % genéricos
Baratos de larga duración	227	2,50 €	Todos de larga duración y 20 años de antigüedad media (simvastatina, losartán, ramipril)
Combinaciones de dos principios activos	108	36,19 €	97 de 108 llevan dos principios activos
Anticoagulantes con ficha compleja	77	42,07 €	71 de 77 son anticoagulantes; 18,7 tablas de media
Inyectables no genéricos	51	48,80 €	Todos inyectables, ninguno genérico y 36 insulinas
Metformina barata de dosis alta	29	1,86 €	Todo metformina, con 902 mg de dosis media

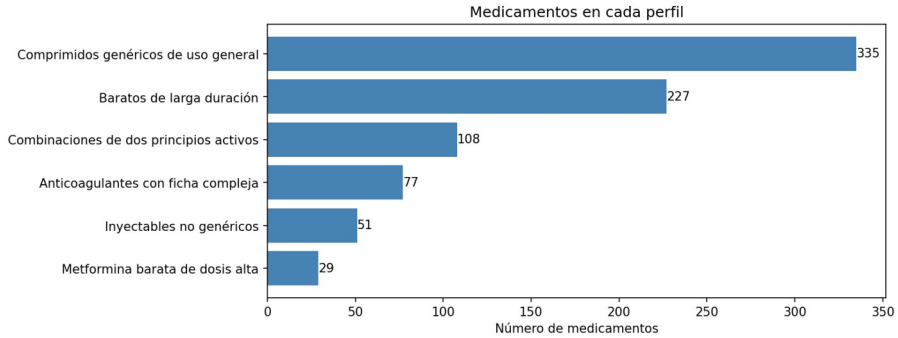


Figura 4. Número de medicamentos de cada perfil.

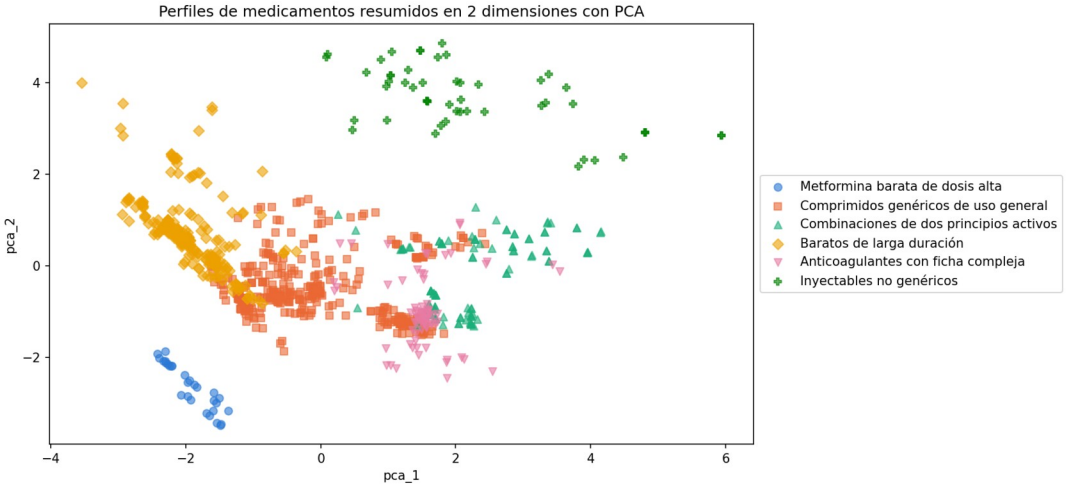


Figura 5. Perfiles resumidos en 2 dimensiones con PCA, que conserva el 38,5 % de la información.

Pantallazos de la ejecución

Estas capturas muestran el código de los cuadernos y la salida que obtuvo al ejecutarse.

h4.ipynb

[169]

```
# ... (celda abreviada: se muestran las últimas líneas)
print("Filas que todavia tienen una barra:", df_clean["dosis"].str.contains("/").sum())


# PASO 14: nos quedamos con el numero de la dosis, que es lo que hay antes del espacio: "50 mg" -> 50
df_clean["dosis_numero"] = df_clean["dosis"].str.split(" ").str[0].astype(float)

# Miramos como ha quedado
df_clean[["nombre", "dosis", "dosis_numero", "dosis_numero_2", "dosis_por_ml"]].head(10)
```

```
Filas que no acaban en mg: 0
Filas que todavia tienen una barra: 0
```

Captura 1. HU-4: comprobación de la columna de dosis (celda abreviada).

h4.ipynb

[177]

```
# Ahora repetimos exactamente lo mismo para todas las columnas numéricas con un bucle
columnas_numericas = [
    "precio_pvp_iva",
    "palabras_seccion_4_4",
    "numero_tablas",
    "indicador_riesgo_severo",
    "num_excipientes",
    "dosis_numero",
]

for columna in columnas_numericas:
    q1 = df_clean[columna].quantile(0.25)
    q3 = df_clean[columna].quantile(0.75)
    iqr = q3 - q1
    limite_inferior = q1 - 1.5 * iqr
    limite_superior = q3 + 1.5 * iqr
    outliers = df_clean[(df_clean[columna] < limite_inferior) | (df_clean[columna] > limite_superior)]
    print(columna, "-> límite superior:", limite_superior, "| outliers:", len(outliers))
```

```
precio_pvp_iva -> límite superior: 63.582499999999996 | outliers: 36
palabras_seccion_4_4 -> límite superior: 2787.0 | outliers: 62
numero_tablas -> límite superior: 6.0 | outliers: 121
indicador_riesgo_severo -> límite superior: 28.5 | outliers: 4
num_excipientes -> límite superior: 6.0 | outliers: 1
dosis_numero -> límite superior: 172.5 | outliers: 52
```

Captura 2. HU-4: detección de outliers con el método IQR.

h5.ipynb

[1]

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, adjusted_rand_score
from sklearn.decomposition import PCA

# dtype indica que nregistro y cn son texto, porque son códigos y no cantidades
df = pd.read_csv("../data/HU4_medicamentos_diabetes_limpio.csv", dtype={"nregistro": str, "cn": str})

# Al leer un CSV las fechas llegan como texto, así que volvemos a pasar estado_aut a fecha
df["estado_aut"] = pd.to_datetime(df["estado_aut"])

print(df.shape)
print("Nulos:", df.isna().sum().sum())
```

```
(827, 28)
Nulos: 0
```

Captura 3. HU-5: carga del CSV limpio de la HU-4.

h5.ipynb

[5]

```
valores_k = [2, 3, 4, 5, 6, 7, 8, 9, 10]
inercias = []
silhouettes = []

for k in valores_k:
    modelo_prueba = KMeans(n_clusters=k, random_state=42, n_init=50)
    # entrena y devuelve el grupo de cada medicamento
    grupos = modelo_prueba.fit_predict(X_escalado)

    inercia = modelo_prueba.inertia_
    silhouette = silhouette_score(X_escalado, grupos)

    inercias.append(inercia)
    silhouettes.append(silhouette)
    print("K =", k, "| inercia:", round(inercia), "| silhouette:", round(silhouette, 3))
```

```
K = 2 | inercia: 8775 | silhouette: 0.211
K = 3 | inercia: 7603 | silhouette: 0.235
K = 4 | inercia: 6693 | silhouette: 0.255
K = 5 | inercia: 5829 | silhouette: 0.286
K = 6 | inercia: 5005 | silhouette: 0.292
K = 7 | inercia: 4492 | silhouette: 0.276
K = 8 | inercia: 4108 | silhouette: 0.292
K = 9 | inercia: 3822 | silhouette: 0.298
K = 10 | inercia: 3587 | silhouette: 0.306
```

Captura 4. HU-5: inercia y silhouette para K de 2 a 10.

h5.ipynb

[8]

```
modelo = KMeans(n_clusters=6, random_state=42, n_init=50)

# Guardamos en una columna nueva el grupo (de 0 a 5) de cada medicamento
df["cluster"] = modelo.fit_predict(X_escalado)

print("Silhouette final:", round(silhouette_score(X_escalado, df["cluster"]), 3))
df["cluster"].value_counts().sort_index()
```

```
Silhouette final: 0.292
cluster
0     29
1    335
2    108
3    227
4     77
5     51
Name: count, dtype: int64
```

Captura 5. HU-5: modelo final y número de medicamentos de cada grupo.

h5.ipynb

[9]

```
# Media de cada columna en cada grupo
# groupby separa las filas por grupo y mean calcula la media de cada columna
medias = df.groupby("cluster")[columnas_modelo].mean().round(2)
medias.T
```

cluster	0	1	2	3	4	5
precio_pvp_iva	1.58	14.44	44.43	4.25	38.10	62.87
palabras_seccion_4_4	853.55	743.67	1308.28	1868.65	1854.00	793.00
numero_tablas	1.10	1.42	4.52	1.24	18.74	3.84
indicador_riesgo_severo	3.07	10.88	12.62	8.97	18.39	9.82
num_excipientes	0.21	1.98	1.44	2.02	2.61	2.80
dosis_numero	901.72	66.86	40.80	35.43	53.67	3.84
num_principios_activos	1.00	1.00	1.90	1.00	1.00	1.12
antiguedad_anios	11.62	13.07	7.94	20.23	6.04	14.98
generico	1.00	0.84	0.47	0.85	0.75	0.00
comercializado	0.59	0.82	0.94	0.73	0.95	0.75
afecta_conduccion	0.00	0.54	0.90	0.01	0.03	0.96
tratamiento_larga_duracion	0.00	0.05	0.00	1.00	0.00	0.65
es_inyectable	0.00	0.00	0.00	0.00	0.00	1.00

Captura 6. HU-5: media de cada columna en cada grupo.

h5.ipynb

[11]

```
# Diccionario con número de grupo y nombre
nombres_grupos = {
    0: "Metformina barata de dosis alta",
    1: "Comprimidos genéricos de uso general",
    2: "Combinaciones de dos principios activos",
    3: "Baratos de larga duración",
    4: "Anticoagulantes con ficha compleja",
    5: "Inyectables no genéricos",
}

# .map traduce cada número usando el diccionario
df["perfil"] = df["cluster"].map(nombres_grupos)

df["perfil"].value_counts()
```

```
perfil
Comprimidos genéricos de uso general      335
Baratos de larga duración                 227
Combinaciones de dos principios activos   108
Anticoagulantes con ficha compleja         77
Inyectables no genéricos                   51
Metformina barata de dosis alta            29
Name: count, dtype: int64
```

Captura 7. HU-5: nombres de los grupos y número de medicamentos de cada uno.

h5.ipynb

[13]

```
pca = PCA(n_components=2)
puntos = pca.fit_transform(X_escalado)

# primera columna de puntos
df["pca_1"] = puntos[:, 0]
 # segunda columna de puntos
df["pca_2"] = puntos[:, 1]

# explained_variance_ratio_ dice cuánta información conserva cada eje
informacion = pca.explained_variance_ratio_.sum() * 100
print("Información que conserva el dibujo:", round(informacion, 1), "%")
```

```
Información que conserva el dibujo: 38.5 %
```

Captura 8. HU-5: cálculo de PCA para dibujar los grupos.

h5.ipynb

[15]

```
# Borramos las columnas que solo necestimos dentro del cuaderno
df_salida = df.drop(columns=["antiguedad_anios", "es_inyectable", "pca_1", "pca_2"])

df_salida.to_csv("../data/HU5_medicamentos_diabetes_clusters.csv", index=False, encoding="utf-8-sig")
```

Captura 9. HU-5: generación del CSV final.

Discusión de los resultados

¿Se comporta el sistema como esperábamos? Sí. Los dos cuadernos se ejecutan de principio a fin y generan los CSV previstos. Las comprobaciones dan el resultado esperado y los perfiles tienen sentido: el modelo ha reunido 71 anticoagulantes en un grupo sin conocer los principios activos, solo con las 13 columnas numéricas.

¿Qué factores han afectado a los resultados?

- **El cruce con el Nomenclátor** nos hizo perder el 47 % de las filas. De esas 742, 632 no están comercializadas y 110 sí, así que los resultados solo valen para los medicamentos que aparecen en el Nomenclátor.

- **La búsqueda de la HU-1** se hizo por la palabra "diabetes" en la sección 4.1. Por eso también hay medicamentos que solo la mencionan, como los de simvastatina o los anticoagulantes.
- **La elección de las columnas** condiciona los grupos. Por ejemplo, `es_inyectable` y `tratamiento_larga_duracion` pesan mucho en dos de los perfiles.
- **Los datos tienen limitaciones.** No sabemos cuántas unidades trae cada presentación, así que el precio no es un coste por dosis. `dosis_numero` mezcla dosis por unidad y por ml, y en las combinaciones solo refleja la primera. Tampoco hemos comprobado si los 3 medicamentos con 0 palabras en la sección 4.4 (VIPIDIA) la tienen en otro formato.

¿Ha habido algún comportamiento inesperado? Sí, en tres puntos:

- En una prueba aparte, con `n_init=10` el resultado cambiaba según la semilla. Al subirlo a 50, las semillas probadas dan siempre los mismos grupos.
- El silhouette no sube de forma continua: baja en K = 7 y vuelve a subir después. Por eso no bastaba con elegir el valor más alto.
- El silhouette final (0,292) es bajo. Los grupos existen, pero se solapan, y algunos medicamentos quedan cerca de la frontera entre dos perfiles, como se ve en la Figura 5.

Conclusión

Resumen del proceso

Partimos de un dataset con 1569 medicamentos que no estaba listo para un modelo. En la HU-4 lo analizamos, limpiamos los nulos, los tipos y la dosis, revisamos los outliers y obtuvimos un CSV con 827 medicamentos y 0 nulos. En la HU-5 preparamos 13 columnas numéricas, las escalamos, elegimos K = 6 y entrenamos un K-Means, cuyos grupos analizamos y nombramos.

Principales logros

- **Dataset preparado:** 827 medicamentos y 28 columnas, sin nulos y con las dosis en un único formato.
- **Modelo estable:** en la prueba aparte con otras 7 semillas salen siempre los mismos 6 grupos.
- **Perfiles interpretables:** van desde la metformina barata de dosis alta (mediana de 1,86 €) hasta los inyectables no genéricos (48,80 €).
- **Flujo reproducible:** los cuadernos generan los CSV `HU4_medicamentos_diabetes_limpio.csv` y `HU5_medicamentos_diabetes_clusters.csv`, listos para las siguientes etapas del proyecto.

El punto más débil es la separación entre grupos (silhouette de 0,292). Por eso, los perfiles sirven para orientarse en el catálogo, pero no son categorías oficiales.